

Designing Tool Interfaces

A tool is only as good as the way you describe it.

IN THIS LECTURE,

1 Tools are Claude's hands

2 The description is how Claude picks

3 Anatomy of a great description

4 Naming to kill ambiguity

5 Split vs consolidate

6 The cost of a vague tool

Tools Are Claude's Hands

Talk is cheap — tools let Claude act.

A tool is a function Claude can call to DO something

- ▶ Look up an order, send an email, query a database
- ▶ Without tools, Claude can only talk
- ▶ Each tool is one action Claude can reach for



🔗 Remember the loop from 1.1 — Claude asks for a tool, your code runs it. A tool is that thing Claude can ask for.

The Description Is How Claude Picks

Claude can't see your code — only your description.



Claude selects a tool from its name + description

- ▶ It cannot see inside your function
- ▶ A vague description means the wrong tool, or no tool
- ▶ Like a shopkeeper reading the label on each box



The description is the selection mechanism — the exam tests this. Write it for Claude, not for yourself.

Anatomy of a Great Description

Say what it does, when to use it, and what to pass.



What it does

The action, in one clear line



When to use it

And when NOT to use it



Each parameter

Explained in plain words



A short example

One tiny sample call



'When to use it' is the part people skip — and it's what stops Claude reaching for the wrong tool.

Name Tools to Kill Ambiguity

If two tools sound the same, Claude will mix them up.



Vague (confusing)

- ✗ `getData` vs `fetchData`
- ✗ Which one does what?



Distinct (clear)

- ✓ `get_order_by_id`
- ✓ `search_orders_by_customer`



Overlapping names and descriptions are a top cause of wrong-tool errors. Make each tool's job obvious and non-overlapping.

Split vs Consolidate

One giant tool, or many small ones?

Size tools around the choices Claude has to make

- ▶ Split when jobs are distinct — clearer selection
- ▶ Consolidate steps that always go together — fewer round-trips
- ▶ Avoid both extremes: too many tiny tools, or one do-everything tool



Too many tiny tools overwhelm selection; one do-everything tool becomes too vague to describe.

Worked Example: Booking Travel

The same feature, designed two ways.



One mega-tool

- ✗ `handle_travel(action, ...)`
- ✗ Claude must guess the 'action' string



Three clear tools

- ✓ `search_flights · book_flight`
- ✓ `cancel_booking` — each one job



Distinct actions → distinct tools. Claude picks `book_flight` unambiguously instead of guessing an 'action' value.

The Cost of a Vague Interface

Bad design doesn't crash — it quietly misbehaves.



Failures are silent, not loud

- ▶ Wrong tool chosen, or wrong parameters filled in
- ▶ Or Claude gives up and just answers in text
- ▶ Shows up as subtly wrong behaviour in production



You won't see a neat error — you'll see subtly wrong behaviour. Invest in the interface before the logic.

Key Takeaways

Tool design, in three lines.

1

Description = the interface

Claude picks tools from their name + description — write for Claude.

2

Clear and distinct

Say what, when, params, example; distinct names kill ambiguity.

3

Right-size your tools

Split distinct actions, consolidate steps that always go together.



End of Lecture 2.1 — Designing Tool Interfaces

Structured Error Responses

When a tool fails, your error message is Claude's only clue.

IN THIS LECTURE,

1 Errors are Claude's window into failure

2 The isError flag

3 Error categories

4 isRetryable

5 Empty result vs real failure

6 Writing a useful error

Errors Are Claude's Window Into Failure

'Something went wrong' tells Claude nothing.



Useless error

- ✗ 'Booking failed.'
- ✗ Claude is stuck — nothing to act on



Actionable error

- ✓ 'Payment failed: insufficient balance.'
- ✓ 'Refund in 3 days. Try another method.'

? Claude can only act on what the error says. Vague errors force it to guess; specific errors let it recover.

The isError Flag

One boolean that says 'I ran, but I failed.'

Return `isError: true` inside a successful response

- ▶ So Claude SEES the failure and can react to it
- ▶ Don't throw a raw exception — Claude handles that poorly
- ▶ A readable failure, not a crash



A thrown exception becomes a low-level protocol error. Return `isError: true` with details instead.

Error Categories

Not all failures are the same kind of failure.



validation

Bad input — fix it and retry



auth / permission

Not allowed — escalate



not_found

Nothing there to return



rate_limit / transient

Service busy — wait, then retry



Tag every error with `errorCategory` — each kind implies a different next move for Claude.

isRetryable: Should Claude Try Again?

The single most useful field in your error.



One boolean that stops blind retry loops

- ▶ `isRetryable: true` → transient / rate_limit (wait, retry)
- ▶ `isRetryable: false` → validation / auth / business (pointless)
- ▶ A 'business' error — like a refund over the limit — is never retryable



Retrying a validation or business error just burns calls and time — the answer will be 'no' every time.

Empty Result vs Real Failure

'Found nothing' is a success, not an error.



Success, but empty

✓ isError: false, resultCount: 0

✓ A real answer: 'no orders found'



Access failed

✗ isError: true, retryable

✗ 'Couldn't reach the database'



The canonical Domain-2 trap: an agent retrying a successful empty query. Empty is an answer; failure is not.

Writing a Useful Error Message

What was attempted · what failed · what to do next.

Say enough to recover — nothing that leaks

- ▶ Good: 'Couldn't fetch order #4471: DB timed out after 5s. Transient — safe to retry.'
- ▶ Bad: a bare 'Error.'
- ▶ Never return a raw stack trace — it leaks your internals

Sanitize: *Wrap 'no such table: users_v2' as 'The requested resource could not be accessed.'*



A raw stack trace confuses Claude and reveals your database and structure. Mask it.

Key Takeaways

Structured errors, in three lines.

1

Return, don't throw

isError: true with details is Claude's only clue — a thrown exception isn't.

2

Category + retryable

Tag errorCategory and isRetryable so Claude fixes, waits, escalates, or stops.

3

Empty is not failure

isError: false with count 0 is a success — never retry it as a failure.



End of Lecture 2.2 — Structured Error Responses

Tool Distribution & tool_choice

More tools isn't better — the right tools, and sometimes a forced choice.

IN THIS LECTURE,

1 The too-many-tools problem

2 Scoping tools to the job

3 The tool_choice lever

4 The four modes

5 When to use each

6 A worked example

The Too-Many-Tools Problem

Give Claude 60 tools and watch it fumble.



60 tools

- ✗ Selection accuracy drops
- ✗ Overlapping tools confuse it

VS



6 sharp tools

- ✓ Faster, more accurate choices
- ✓ The right few for the job



Overloading the toolset is a silent quality killer — Claude picks slower and less accurately.

Scope Tools to the Job

Every agent gets only the tools its role needs.

Don't hand every agent every tool

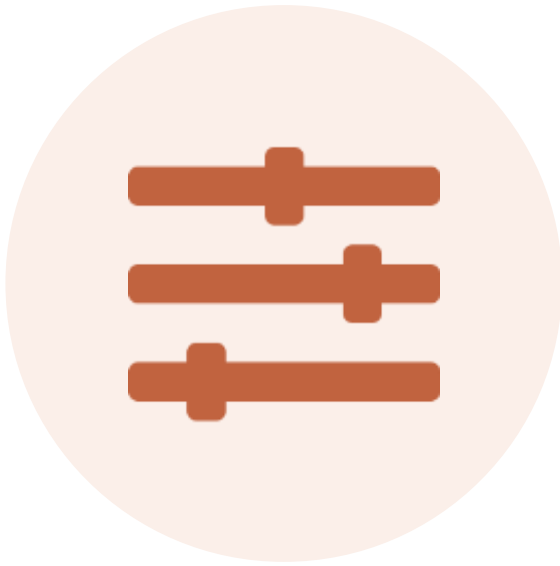
- ▶ A read-only reporter needs read tools only
- ▶ A refund agent needs refund tools
- ▶ Fewer, role-appropriate tools → sharper choices



🔗 Same scoping idea from 1.3 — a subagent's `allowedTools` should list only what its role needs.

Enter tool_choice: the Control Lever

By default Claude decides. Sometimes you can't allow that.



Scoping sets the menu; tool_choice sets the order

- ▶ Scoping controls WHICH tools exist
- ▶ tool_choice controls WHETHER and WHICH gets called now
- ▶ Take the decision back when you need certainty



Scoping = the menu. tool_choice = whether Claude may order, must order, or must order one specific dish.

The Four Modes

auto · any · tool · none.



auto

Claude decides (the default)



any

Must use some tool — Claude picks which



tool

Must use THIS named tool, every time



none

No tools this turn — text only



Memorise all four — the exam asks you to pick the right mode for a scenario.



'any' and 'tool' (forced use) can't be combined with extended thinking.

When to Use Each

Match the mode to how much certainty you need.

From free choice to non-negotiable

- ▶ auto → ordinary chat assistants
- ▶ any → 'must act, you choose how'
- ▶ tool → guaranteed output / always-save
- ▶ none → ask without acting, or a text-only turn



💡 Reach for 'tool' when a step is non-negotiable — e.g. a research agent that must call `save_report` before finishing.

Worked Example: Always Save the Report

Turning 'usually saves' into 'always saves'.



On auto (risky)

- ✗ Sometimes replies in text
- ✗ Findings are lost



Forced tool (safe)

- ✓ `tool_choice: { type:'tool', name:'save_report' }`
- ✓ Saves every single run



Forcing a specific tool is the standard trick for guaranteed structured output — we'll reuse it in Domain 4.

Key Takeaways

Distribution & tool_choice, in three lines.

1

Fewer, scoped tools

A giant toolbox hurts selection — give each role only what it needs.

2

Know the four modes

tool_choice: auto (decide), any (some tool), tool (this tool), none (no tools).

3

Force when it must happen

Use 'tool' for always-save and guaranteed structured output.



End of Lecture 2.3 — Tool Distribution & tool_choice

MCP Server Integration

One standard plug that connects Claude to almost anything.

IN THIS LECTURE,

- 1 What MCP is
- 2 Tools vs resources
- 3 Connecting a server
- 4 Keeping secrets safe

- 5 The three scopes
- 6 Precedence
- 7 Multiple servers

What Is MCP?

USB-C for AI tools.

A universal standard for connecting Claude to external systems

- ▶ Before MCP: a custom connector for every single tool
- ▶ With MCP: one standard plug — any MCP server just works
- ▶ Build the server once; Claude plugs in



Like USB-C replacing every proprietary charger — one plug for GitHub, databases, Slack, and more.

Tools vs Resources

Verbs you can call, and data you can pull in.



Tools = verbs

- ✓ Actions Claude calls
- ✓ `create_issue`, `run_query`



Resources = nouns

- ✓ Read-only data / context
- ✓ A file, a doc, a schema — pull in with `@`



Tools DO things; resources INFORM Claude. In Claude Code you reference a resource by typing `@`.

Connecting a Server

A small JSON entry — command or URL.



Each server is one entry under mcpServers

- ▶ stdio → a local program Claude launches (command + args)
- ▶ http → a remote server (a url)
- ▶ 'Transport' = HOW Claude talks to it (local vs remote)

Config: `'docs': { 'type':'http', 'url':'https://.../mcp' }`



Transport (stdio vs http) is separate from WHERE the config lives — that's the next few slides.

Keep Secrets Out of the Config

Reference secrets by name — never paste them in.

Use `${VAR}` expansion so keys live in your environment

- ▶ The API key stays out of the committed file
- ▶ `${VAR:-default}` gives a fallback value
- ▶ Works in command, args, env, url, and headers

Safe: `'env': { 'API_TOKEN': '${GITHUB_TOKEN}' }`



⚠️ A key hard-coded in a shared `.mcp.json` gets committed to git — a real leak. `${VAR}` keeps it off disk.

The Three Scopes

Just you, your team, or all your projects.

local

~/.claude.json · only you, this project (default)



project

.mcp.json · committed, the whole team



user

~/.claude.json · you, across all projects



Picking the wrong scope is the #1 MCP mistake — a personal token in 'project' scope leaks to the team.

Precedence & Inheritance

When the same server exists twice, who wins?

local > project > user



- ▶ Same-named server → the higher scope wins
- ▶ Your local override beats the team's shared one
- ▶ Run in a sub-folder? Parent .mcp.json files merge in too



Knowing this order saves a lot of 'why is it using the wrong server?' confusion.

Multiple Servers

Many servers, one session — keep them tidy.

Run several servers at once

- ▶ GitHub + database + browser automation, together
- ▶ Check connection status with `/mcp`
- ▶ Name them clearly — github, stripe-prod, not server1



Name servers like variables. You'll be reading these names months later — treat them that way.

Key Takeaways

MCP integration, in three lines.

1

One standard plug

MCP servers offer tools (actions) and resources (data via @).

2

Scopes + secrets

local > project > user; use \${VAR} to keep keys out of git.

3

Many servers, tidy

Run several at once; check with /mcp and name them clearly.



End of Lecture 2.4 — MCP Server Integration

Built-in Tools

The toolkit Claude Code ships with — and how to pick the right one.

IN THIS LECTURE,

1 The six built-in tools

2 Grep vs Glob

3 Read, Edit, Write

4 Read-before-Edit

5 Prefer built-ins over Bash

6 Explore incrementally

The Six Built-in Tools

A small toolkit, each with one clear job.



Read

Open and view a file



Write

Create or overwrite a file



Edit

Change part of a file



Bash

Run shell commands



Grep

Search inside files



Glob

Find files by name



These are the tools you'll see most in Claude Code — each has one job, and the wrong pick wastes tokens and time.

Grep vs Glob: the #1 Confusion

Search inside files, or find files by name.



Grep = contents

- ✓ Search **INSIDE** files
- ✓ Find where `processOrder` is called



Glob = filenames

- ✓ Match file paths / names
- ✓ Find every `*.test.js`



The exam tests this directly and penalises mix-ups: contents → Grep, filenames → Glob.

Read, Edit, Write

See a file, change part of it, or replace the whole thing.



Read

View the file



Edit

Precise change — exact string replace



Write

Create / overwrite the whole file



For a small change use Edit, not Write — Edit shows a clean diff; Write rewrites everything.

The Golden Rule: Read Before Edit

Claude must see a file before it can change it.

Edit requires a prior Read

- ▶ The target text must match exactly and be unique
- ▶ Not unique? Widen the surrounding text, or use `replace_all`
- ▶ Can't get a clean match? Read + Write is the last-resort fallback

⚠️ A non-unique or stale match fails — pin one spot by widening the text.

❓ Read + Write is the fallback, not the default. Prefer a precise Edit.



Prefer Built-ins Over Bash

Use Read, not cat. Use Grep, not grep.



Built-in tools

- ✓ Read / Grep / Glob / Edit
- ✓ Better permissions, clearer trail, cacheable



Bash one-liners

- ✗ cat / grep / find / sed
- ✗ Extra permission prompts that can't be cached



Save Bash for what it's uniquely good at — actually running things, like your tests or a build.

Explore Incrementally

Don't read the whole library — use the catalogue.



Find first, then read only what matters

- ▶ Use Grep / Glob to locate the few files that matter
- ▶ Then Read only those
- ▶ Never read every file upfront

🔍 Reading everything blows your context budget — locate first, read narrowly.

🔗 Managing that context budget is Domain 5 — the good habit starts here.

Key Takeaways

Built-in tools, in three lines.

1

Six tools, clear jobs

Read/Write/Edit/Bash/Grep/Glob —
Grep = contents, Glob = filenames.

2

Read before Edit

Exact + unique match, else
replace_all or the Read+Write
fallback.

3

Built-ins + incremental

Prefer built-ins over Bash; locate first,
don't read everything.



End of Lecture 2.5 — Built-in Tools